

CE802 2017 - Worked Solutions

Machine Learning and Data Mining

This document gives complete worked solutions for the uploaded 2017 CE802 paper. The aim is not to present a bare answer key; instead, every question is solved by showing the underlying principle, the relevant formula, and the reasoning that turns the formula into the final conclusion.

Notation is kept simple: class labels are written as +1 / -1; supports are given both as counts and as proportions where useful; and all numerical calculations are rounded only at the end so that intermediate rounding does not change the final answer.

Question 1

1(a) Why support vector classifiers can be viewed as instance-based learners

A support vector classifier does not summarise the training set by a single closed-form prototype such as a mean vector. Instead, prediction for a new example x is obtained by comparing x with a subset of the training instances themselves - the **support vectors**. This is why SVCs can be interpreted as **instance-based** learners.

The similarity between the query point and each retained training instance is measured by a **kernel** $K(x_s, x)$. Examples include the linear kernel $x_s^T x$, polynomial kernels, and the radial-basis-function kernel $\exp(-\gamma \|x_s - x\|^2)$. Only training instances with non-zero Lagrange multiplier α_s are retained; these are exactly the support vectors.

Their contribution to prediction is through the decision function

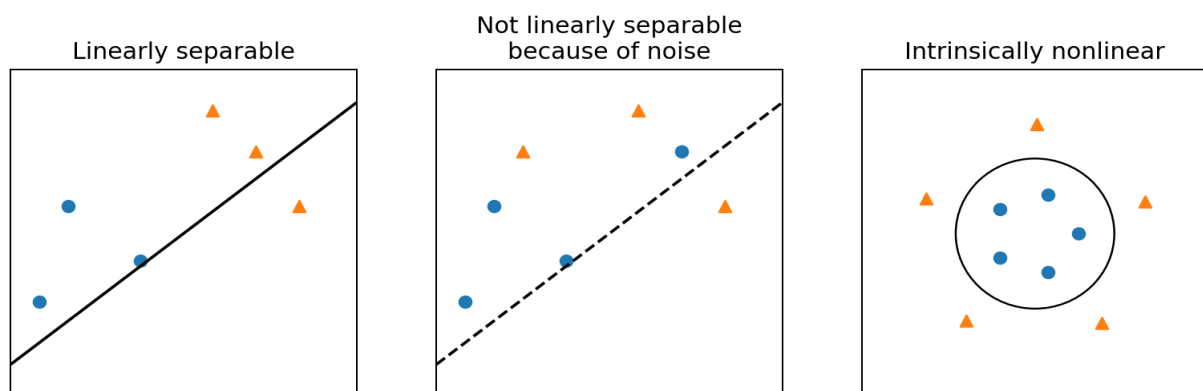
$$h(x) = [\sum_{s \in S} \alpha_s r^{(s)} K(x^{(s)}, x)] - b.$$

Each support vector casts a weighted vote. The sign of $h(x)$ determines the class, while the magnitude indicates how confidently the point is placed relative to the decision surface.

1(b) Linear separability

A set of labelled examples is **linearly separable** if there exists at least one hyperplane that classifies every training point correctly with zero training error. In two dimensions the hyperplane is just a straight line.

The three requested situations are illustrated below. In the noisy case a straight line would work except for one or a few inconsistent points. In the intrinsically non-linear case no straight line can separate the classes at all; a curved boundary is required.



1(c) Support vectors in Figure 1.1

In a soft-margin SVC, the support vectors are the training points that lie **on the margin** or **inside the margin** (including misclassified points). Looking at Figure 1.1, the dotted contours correspond to the ± 1 margins and the solid contour corresponds to the decision boundary $h(x)=0$.

From the diagram, the points that force the boundary are: **e** and **f** on the -1 margin; **q**, **n**, **o**, and **r** on or very near the +1 margin; and the negative points **k**, **m**, and **p**, which lie on the positive side / inside the positive margin and therefore require the classifier to bend around them. These are the support vectors. Points such as a, b, c, d, g, h, i, j

and I are farther from the margin and therefore have $\alpha = 0$.

Because the figure is hand-drawn, exact equality to the contour is interpreted visually: points touching the margin or clearly lying in the margin band are treated as support vectors.

1(d) Numerical evaluation of the RBF SVC

The support vectors and weights are

$$x^{(2)}=[0.2, 0.8, 0.1], r^{(2)}=+1, \alpha_2=0.25$$

$$x^{(5)}=[-0.5, 0.2, 0.7], r^{(5)}=-1, \alpha_5=1.0$$

$$x^{(8)}=[0.5, 0.8, 0.1], r^{(8)}=+1, \alpha_8=0.5, \text{ with } \gamma = 0.4 \text{ and } b = 0.3.$$

For the new point $x=[-0.5, 0.8, 0.1]$, compute the squared Euclidean distances first.

Pair	Difference vector	$\ x(s)-x\ ^2$	Kernel value $\exp(-0.4 d)$
$x^{(2)}$ vs x	[0.7, 0, 0]	0.49	$\exp(-0.196)=0.8220$
$x^{(5)}$ vs x	[0, -0.6, 0.6]	0.72	$\exp(-0.288)=0.7498$
$x^{(8)}$ vs x	[1.0, 0, 0]	1.00	$\exp(-0.4)=0.6703$

Now substitute into the decision function:

$$h(x) = 0.25(+1)(0.8220) + 1.0(-1)(0.7498) + 0.5(+1)(0.6703) - 0.3$$

$$= 0.2055 - 0.7498 + 0.3352 - 0.3 = -0.5091 \text{ (approximately).}$$

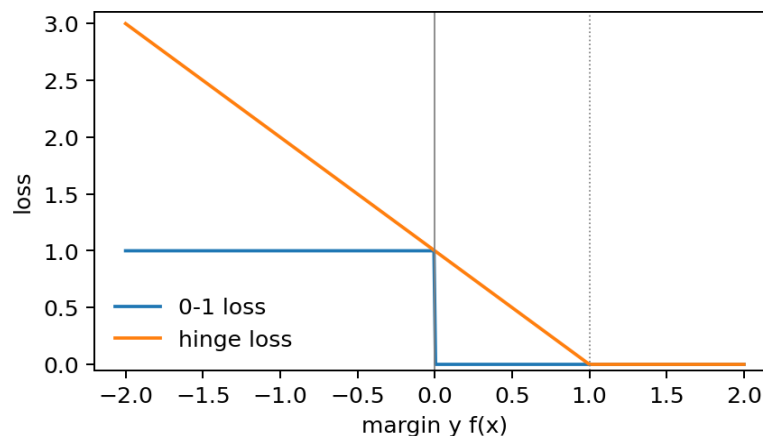
Since $h(x) < 0$, the new example is classified as **class -1**. The negative sign is not marginal: the point lies comfortably on the negative side of the decision surface.

1(e) 0-1 loss versus hinge loss

For a signed margin $z = y f(x)$, the two losses are

$$0-1 \text{ loss: } L_{01}(z)=1 \text{ if } z<0, \text{ and } 0 \text{ otherwise.}$$

$$\text{hinge loss: } L_{\text{hinge}}(z)=\max(0, 1-z).$$



Three disadvantages of 0-1 loss compared with hinge loss are:

- Non-convex optimisation.** Minimising 0-1 loss leads to a difficult combinatorial problem with many poor local minima. Hinge loss is convex, which is why SVM training is tractable.
- Non-differentiability / flat regions.** Away from the decision boundary the 0-1 loss gives no gradient information at all. Hinge loss still provides a useful penalty until the margin reaches 1.
- No margin encouragement.** Under 0-1 loss, any correctly classified point receives exactly the same loss, whether it is barely correct or very confidently correct. Hinge loss explicitly rewards a large margin, which usually improves generalisation.

Question 2

Movies legend used below

A = Aliens, I = Inception, Av = Avatar, T = Titanic, S = Scarface, G = Goodfellas.

2(a) Frequent itemsets by Apriori (minimum support count = 3)

There are 5 transactions, so the minimum support threshold 60% corresponds to a support count of 3. Apriori proceeds level by level: first all 1-itemsets, then candidate 2-itemsets generated from the frequent 1-itemsets, then candidate 3-itemsets generated from the frequent 2-itemsets, and so on.

1-itemset	Support count	Frequent?
{A}	5	Yes
{I}	3	Yes
{Av}	4	Yes
{T}	1	No
{S}	2	No
{G}	3	Yes

Therefore $L1 = \{\{A\}, \{I\}, \{Av\}, \{G\}\}$.

Candidate 2-itemset	Support count	Outcome / reason
{A,I}	3	Retain
{A,Av}	4	Retain
{A,G}	3	Retain
{I,Av}	3	Retain
{I,G}	1	Discard - insufficient support
{Av,G}	2	Discard - insufficient support

Only one 3-itemset survives candidate generation. The triples {A,I,G}, {A,Av,G}, and {I,Av,G} are not even generated because they contain the non-frequent 2-subsets {I,G} and/or {Av,G}; this is exactly the Apriori pruning step.

Candidate 3-itemset	Reason it is considered / pruned	Support count	Outcome
{A,I,Av}	all 2-subsets frequent	3	Retain
{A,I,G}	contains non-frequent subset {I,G}	-	Pruned by Apriori
{A,Av,G}	contains non-frequent subset {Av,G}	-	Pruned by Apriori
{I,Av,G}	contains non-frequent subsets {I,G} and {Av,G}	-	Pruned by Apriori

Hence the complete family of frequent itemsets is:

$$L = \{\{A\}, \{I\}, \{Av\}, \{G\}, \{A,I\}, \{A,Av\}, \{A,G\}, \{I,Av\}, \{A,I,Av\}\}.$$

2(b) Association rules that predict Inception

For a rule $X \Rightarrow I$ we use

$$\text{confidence}(X \Rightarrow I) = \text{support}(X \cup \{I\}) / \text{support}(X),$$

$$\text{lift}(X \Rightarrow I) = \text{confidence}(X \Rightarrow I) / \text{support}(I).$$

Here $\text{support}(I) = 3/5 = 0.6$.

Rule	Support count numerator	Confidence	Lift
{A} => {I}	supp(A,I)=3	3/5 = 0.60	0.60 / 0.60 = 1.00

Rule	Support count number	Confidence	Lift
$\{Av\} \Rightarrow \{I\}$	$\text{supp}(Av,I)=3$	$3/4 = 0.75$	$0.75 / 0.60 = 1.25$
$\{A,Av\} \Rightarrow \{I\}$	$\text{supp}(A,Av,I)=3$	$3/4 = 0.75$	$0.75 / 0.60 = 1.25$

Interpretation: knowing only that a viewer watched Aliens does not change the chance of Inception (lift = 1). By contrast, Avatar alone, or the pair {Aliens, Avatar}, increases the chance of Inception by a factor of 1.25 relative to the baseline support of Inception.

2(c) Similarity table and agglomerative hierarchical clustering

The similarity between two movies is the number of viewers who watched both. The similarity between two groups is the similarity of the **least similar** cross-group pair, so this is the similarity analogue of **complete linkage**. The pairwise similarity table is:

	A	I	Av	T	S	G
A	-	3	4	1	2	3
I		-	3	0	0	1
Av			-	1	1	2
T				-	1	1
S					-	2
G						-

Now merge the most similar clusters step by step:

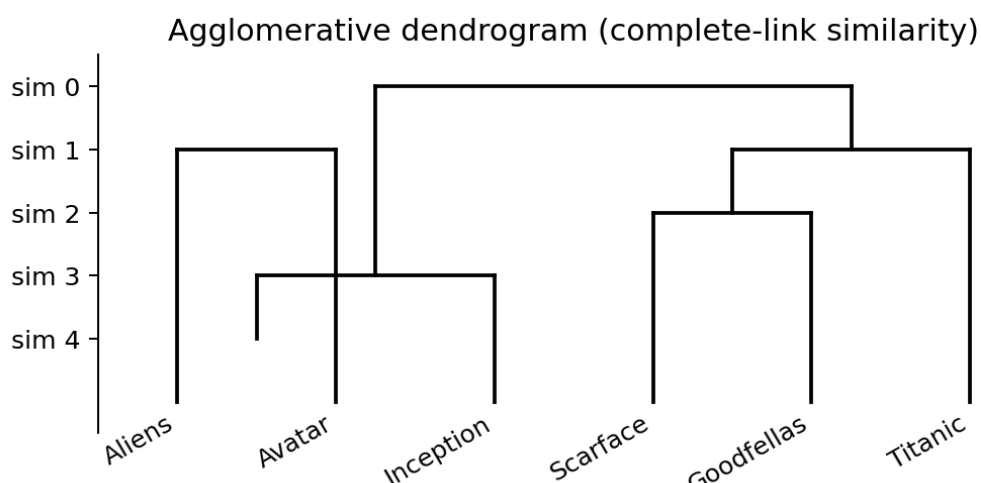
Step 1: merge {A} and {Av} because $\text{sim}(A,Av)=4$, producing cluster $C1=\{A,Av\}$.

Step 2: compare C1 with I. Complete-link similarity is $\min(\text{sim}(A,I), \text{sim}(Av,I)) = \min(3,3) = 3$, so merge to obtain $C2=\{A,Av,I\}$.

Step 3: among the remaining singletons, $\text{sim}(S,G)=2$ is the largest, so merge to obtain $C3=\{S,G\}$.

Step 4: merge Titanic with C3 because $\text{sim}(T,\{S,G\}) = \min(\text{sim}(T,S), \text{sim}(T,G)) = \min(1,1)=1$, giving $C4=\{T,S,G\}$.

Step 5: final merge. The similarity between $C2=\{A,Av,I\}$ and $C4=\{T,S,G\}$ is the minimum of all cross-pair similarities. Because $\text{sim}(I,T)=0$ and $\text{sim}(I,S)=0$, the group similarity is 0. So the final merge occurs at similarity 0.



2(d) Can the dendrogram be updated locally after one new transaction?

In general, **no**. A new transaction can change many pairwise movie similarities at once, which may alter the order of early merges. Since each later merge depends on earlier cluster compositions, one local modification is usually not enough; the safest procedure is to recompute the hierarchy from the updated similarity matrix.

Question 3

3(a) Why naive Bayes is usually less prone to overfitting than decision trees

Naive Bayes usually has **high bias but low variance**. It estimates a small number of probabilities - class priors and one conditional distribution per feature per class - and it uses all training cases for each estimate. Decision tree induction is more flexible, but that flexibility comes with higher variance: each split fragments the data, and deep branches may end up fitting accidental quirks of small subsets.

So, for a fixed dataset, overfitting is less likely with naive Bayes because: (i) the independence assumption strongly constrains the hypothesis class; (ii) parameter estimation is count-based and therefore stable; and (iii) there is no recursive partitioning that repeatedly tailors the model to tiny regions of the feature space.

3(b) Training algorithm for a shallow decision tree with a naive Bayes classifier at each leaf

Assume the dataset has m features and we choose a fixed tree depth $k < m$.

Step 1 - build a shallow tree. Starting at the root, choose the best splitting attribute by a standard criterion such as information gain. Recurse only until depth k is reached. Each internal node therefore consumes one feature along its path.

Step 2 - collect the examples arriving at each leaf. For every leaf L , keep the subset D_L of training examples that satisfy the k tests on the path from the root to L .

Step 3 - train a local naive Bayes model at that leaf. Using only D_L , estimate $P(C)$ and the conditional probabilities for the remaining $m-k$ features that were not already used on the path. These probabilities can be smoothed if needed.

Prediction. A new instance first traverses the tree using the k path features. Once it reaches leaf L , the local naive Bayes model at L computes the posterior over classes from the remaining features, and the class with maximum posterior is returned.

A simple practical example is credit approval. The tree might first branch on income band and employment type (two coarse, highly informative features). Within the resulting leaf, a naive Bayes classifier could then combine many weaker features such as age bracket, debt ratio, number of cards, and recent late payments.

3(c) Why the hybrid can outperform both a standard tree and a standard naive Bayes

Compared with a pure decision tree, the hybrid avoids growing many extra levels just to handle weaker evidence. Instead of fragmenting the data repeatedly, it uses a local probabilistic model at each leaf, which lowers variance and can make better use of limited data.

Compared with a pure naive Bayes classifier, the tree removes some of the strongest context-specific dependencies before naive Bayes is applied. This helps because the independence assumption may be badly violated globally, but may become much more reasonable *within a leaf* after conditioning on a few key attributes.

This kind of dataset is particularly suitable when: (i) a small number of features create broad regimes or subpopulations; (ii) inside each regime the remaining features are approximately conditionally independent; and (iii) the full feature set is large enough that a very deep tree would overfit, but a single global naive Bayes model would be too simplistic.

In short: the tree supplies **context selection**, while the leaf-level naive Bayes supplies **robust probabilistic aggregation**. The combination can therefore achieve a better bias-variance trade-off than either component by itself.

End of worked solutions.